

CoreX

Как это работает

Документация для себя: что за процессы, куда идёт сообщение из чата, как выбирается файл, почему змейка стала `snake.py`, а `/test1` — папка. Схема Draw.io лежит рядом: `corex-architecture.drawio` (открыть в `diagrams.net` или VS Code / Cursor с расширением Draw.io).

Сентябрь 2026. Репозиторий на диске: папки `frontend/`, `backend/core/`, `docs/architecture/`.

1. Одной фразой

CoreX — это окно на Windows (Electron + React) и Python-сервер рядом. Ты пишешь в чат. Сервер зовёт локальную модель (Ollama) или онлайн-API. Модель не «болтает в воздухе»: она просит записать файл, поправить строку или запустить скрипт. Python пишет на диск в открытую папку проекта. Редактор открывает этот файл.

Исходники проекта в облако не уезжают. Ключи онлайн-моделей лежат в `%LOCALAPPDATA%\CoreX\secrets\` (файл `ai_online_providers.json`), не в git. Папок `chat/` две, обе скрыты из дерева и git: в установке CoreX — настройки движка (`ai_runtime.json`); в папке проекта — история чата и `project_memory.md`.

2. Три процесса

- Electron — окно, меню, выбор папки, GPU для интерфейса (на гибриде Intel+NVIDIA UI сидит на iGPU).
- Python backend на свободном порту 8000–8099 — HTTP `/api/*` (файлы, настройки) и WebSocket `/ws` (чат, мысли, патчи редактора). Electron читает строку `COREX_READY port=N`.
- Ollama — локальная модель. Системное приложение слушает 11434. Свой сервер CoreX — 11435. На ПК с Intel+NVIDIA к 11434 лучше не цепляться: видеокарту оставляем модели.

Запуск из исходников: `start-corex.bat` (окно + backend). Разработка UI: `start-corex-dev.bat`. Остановка: `stop-corex-runtime.bat`.

3. Что ты видишь в окне

- Дерево файлов — папка проекта. `chat/` не показывают.
- Редактор — вкладки. Когда агент записал файл, приходит `editor_patch` и вкладка переключается на него.
- Чат — запрос, «мысли», статус «Пишу в ...», карточки файлов, «Готово».
- Терминал — кнопка Run, `pip`, интерактивный ввод.

- Настройки — локальная / онлайн модель, интернет (никогда / спросить / всегда), GPU.
- Visio / схема — обзор файлов проекта, не путать с этим Draw.io.

4. Путь одного запроса

Пример: «создай сапёра с игровым окном в /test1».

- Чат шлёт текст по WebSocket в backend.
- ws_server отдаёт задачу в orchestrator.execute_task.
- task_routing решает: это код, вопрос или «привет». Код идёт дальше.
- write_target смотрит /test1: нет расширения → папка. Имя файла по задаче → minesweeper.py. Итог: test1/minesweeper.py.
- file_mentions не прикрепляет содержимое папки и чужие файлы (змею не подсовывают в промпт).
- Если просили сапёра с окном и файла ещё нет — game_gui_upgrade сразу пишет готовый tkinter. 3В сама сапёра не соберёт.
- Иначе цикл: модель отвечает JSON или блоком ```python```. agent_json это разбирает.
- Инструмент write_file / patch_file / view_file / run_file. Путь ещё раз прогоняют через remap_write_path (main.py модели → нужная папка/имя).
- file_service: нет расширения — отказ; заглушка из двух print — отказ; пустое pygame-окно — отказ. Родители папки создаются. Если test1 был ошибочным файлом — его снимают и делают папку.
- Успех → editor_patch в UI, проверка синтаксиса, иногда run_file. Игры с while True / input() не гоняют как дымовой тест.
- 8 ходов подряд без новой записи — стоп (или снова шаблон сапёра). Чтобы Ollama не крутилась зря.

5. /путь в чате — это не «аттач картинки»

Чип над полем ввода просто показывает, что в тексте есть /ссылка. Смысл ссылки зависит от фразы.

- /main.py + «что делает» — только чтение. Файл прикрепляют в промпт, на диск не пишут.
- /main.py + «исправь» — правка этого файла. Другой путь моделью подменить нельзя.
- /test1 + «создай ...» — папка. Внутри новый файл с нормальным именем. Не test1 и не test1/main.py.
- Без / вообще — агент сам выбирает имя: змейка → snake.py, калькулятор → calculator.py.
- Файл без расширения сохранить нельзя. Сообщение: одно название недостаточно.
- Если test1 уже лежит как файл (старая ошибка) — для /test1 это всё равно папка. При записи файл убирают, создают каталог.

6. Инструменты агента

- write_file — новый или полный файл. Предпочтительно для новой программы: один

целый .py за ход.

- `patch_file` — одна строка (replace/delete). Для точечной правки после `view_file`.
- `view_file` — прочитать. Для создания с нуля 3В часто не должна сначала листать пустую папку.
- `append_file` — дописать кусок. Служебный текст из промпта отбрасывается.
- `run_file` / терминал — запуск. `pip` ставится в тот же Python, что и кнопка Run. Для 3.14 `pygame` -> `pygame-ce`.

7. Модели и железо

Рабочий ПК (i7, 16 ГБ, T600 4 ГБ): держимся Qwen 2.5 Coder 3B, контекст 4096. Домашний (32 ГБ, RTX 3050 8 ГБ): можно 7B. Онлайн — запасной канал, если локальный движок лежит, или если ты сам включил online в настройках.

`llm_runtime` собирает `ActiveLlm`: режим `local` или `online`, имя модели, клиент. Для слабой локальной модели промпт ужимают (`local_pipeline_profile`): один файл за ход, JSON не обязателен.

8. Защиты, чтобы 3В не сожрала игру

- Пустое `pygame`-окно и «два `print`» не записываются поверх живого кода.
- Недописанный `tkinter` (нет `mainloop` / `pass` вместо логики) отклоняется.
- Нельзя затереть готовую игру скелетом.
- Синтаксис чинят (отступы, обрыв), логику игры — нет.
- Шаблон сапёра подставляют только если просили сапёра (или в файле уже сапёр). Змейку им не подменяют.

9. Карта папок

<code>frontend/</code>	окно: Electron + React
<code>electron.cjs</code>	старт UI и Python
<code>src/app/</code>	чат, редактор, дерево, настройки
<code>backend/core/</code>	мозг агента (см. лист 4 в Draw.io)
<code>orchestrator.py</code>	цикл задачи
<code>write_target.py</code>	куда писать
<code>file_service.py</code>	диск
<code>game_gui_upgrade.py</code>	шаблон окна-сапёра
<code>chat/ (y CoreX)</code>	<code>ai_runtime.json</code> , лимиты, лог Ollama
<code>{проект}/chat/</code>	история чата, <code>project_memory.md</code> , visio
<code>core_x_skills/</code>	библиотека скиллов
<code>core_x_agents/</code>	персоны пайплайна
<code>docs/architecture/</code>	эта документация и <code>.drawio</code>

10. Как читать схему Draw.io

Файл: docs/architecture/corex-architecture.drawio. Четыре листа:

- 1. Общая схема — ты, окно, backend, диск, Ollama.
- 2. Ход сообщения — 15 шагов от фразы до «Готово».
- 3. Куда писать — развилка /файл vs /папка vs без слэша.
- 4. Модули backend — какой .py за что отвечает.

Открыть: <https://app.diagrams.net> → Open Existing → выбрать файл. Или расширение «Draw.io Integration» в Cursor/VS Code.

11. Частые «почему так»

Почему змейка в snake.py, а не в main.py?

Ты не указал /путь. Система сама берёт имя из задачи. Корневой main.py не затирают.

Почему сапёр в /test1 сначала не записался?

ЗВ не собирает сапёра за 8 ходов. Шаблон должен писаться сразу в test1/minesweeper.py. Если /test1 ошибочно стал файлом без расширения — запись внутрь ломалась. Сейчас имя без точки = папка, лишний файл test1 при записи снимают.

Почему интернет «не гуглит весь проект»?

web_access: never / ask / always. Поиск — короткие сниппеты (DuckDuckGo). Исходники папки на сервер поиска не отправляют.

Где ключи и модели?

Ключи: %LOCALAPPDATA%\CoreX\secrets\. Модели Ollama: либо системная папка .ollama, либо каталог моделей установки CoreX. Предустановки — chat/ai_runtime.json (скрыт).

12. С чего править код, если снова «не туда пишет»

- backend/core/write_target.py — смысл /пути и имя файла.
- backend/core/file_service.py — физическая запись, запрет без расширения, папка вместо файла test1.
- backend/core/orchestrator.py — цикл, fallback окна, editor_patch.
- backend/core/game_gui_upgrade.py — когда подставлять сапёра.
- frontend/src/app/contexts/ChatContext.tsx — приём патча в редактор.